

A Datomic in beam-lisp

An immutable temporal database, built in the language it is written in

2026-08-21 · implemented · status: feature-complete, persistent on redb

1 What this is, from zero

A database where **nothing is ever updated or deleted**.

Changing an attribute does not overwrite anything. It appends two facts: a retraction of the old value and an assertion of the new one, both stamped with the transaction that made the change. The old value does not move to an archive — it stays exactly where it was, marked as no longer current.

Everything interesting follows from that single decision.

Because nothing is deleted	time travel is a filter over data that never went anywhere, not a feature that had to be built
Because a fact is self-contained	adding an attribute to one entity touches nothing else — there is no table shape to migrate
Because a database is a value	a query is a function of that value, not a request to a server, and holding one from an hour ago costs the same as holding an integer
Because queries are data	a program can build one with conj — no string assembly, no injection surface

This document explains each of those, shows the implementation, and ends with the measurements that should decide what storage substrate comes next.

1.1 Where it lives

priv/datom.bl	the public API — one namespace, the whole database
priv/datom/ store.bl	L1: the storage port (six methods)
store-ets.bl	L0: an ETS implementation of it
codec.bl	order-preserving binary keys
index.bl	datoms, and the four indexes
schema.bl	what an attribute means
tx.bl	transactions: tempids, upsert, net effect
db.bl	the database as a value
conn.bl	the one mutable thing
time.bl	as-of / since / history
pull.bl	reading trees out of the graph
query/parse.bl	a query is data, so parsing is validation
query/plan.bl	index selection and clause ordering
query/engine.bl	the join
query/agg.bl	aggregates

240 tests, 494 assertions in test/bl/datom/. Five executable tutorials in examples/datom/.

2 The datom

A fact has five parts:

```
[entity attribute value transaction added?]
[1      :person/name "Ada" 1000001  true]
```

The first three are what you would expect. The last two are what make this different from a key/value store: the transaction says **when**, and **added?** says whether the fact **began or ended**.

That is the whole data model. There are no tables, no rows, no columns, and no schema-shaped storage — an entity is simply whichever datoms happen to share an id.

2.1 Why five fields and not three

With three fields you have a triple store, and you can answer “what is true?”. With five you can also answer “what **was** true, when did it change, and who changed it?” — and you can answer them with the same query engine, because the extra fields are ordinary addressable positions.

```
[?e :person/age ?age]      ; what is true
[?e :person/age ?age ?tx]   ; and which transaction made it so
[?tx :audit/by ?who]        ; and who ran that transaction
```

2.2 Datom identity is all five fields

A subtle and expensive lesson. The index key originally carried `[e a v tx]`, on the reasoning that a datom is identified by what it says and when. But within **one** transaction a fact can be retracted and re-asserted, and those are two different datoms agreeing on all four fields.

Their keys were byte-identical, and the store is a keyed ordered set: whichever was written second silently replaced the first. Unrecoverable loss in a database whose entire premise is that nothing is lost — and invisible, because the surviving datom looked perfectly well-formed.

op now terminates every index key. Its position is deliberate: last, so it separates two otherwise-identical keys without disturbing sort order within a transaction.

3 Any BEAM term is a value

Datomic’s value types are a closed set: string, long, float, instant, uuid, keyword, ref, bytes. That set exists because Datomic runs on the JVM and had to choose a serialization. Want to store a nested map? Serialize it to a string, and give up querying it.

This database runs on the BEAM, where a term is **already** a universal data type with a total order. So the value type is: any BEAM term.

```
(transact! conn
  [{:db/id -1 :doc/payload {:server {:host "localhost" :port 8080}
                                   :retries [100 200 400]
                                   :flags #{:tls :http2}}}))

;; and it is findable BY VALUE, not merely retrievable
(q '[:find ?e :in $ ?v :where [?e :doc/payload ?v]] db
  {:server {:host "localhost" :port 8080}
   :retries [100 200 400]
   :flags #{:tls :http2}})
```

3.1 Storing is easy; indexing is the work

`term_to_binary` round-trips anything. But an index is an order-preserving encoding plus a range scan, and `term_to_binary` orders by serialized **shape** rather than by value. So the codec must project the language’s own order into bytes for every term shape:

tuple	arity first, then elements — Erlang’s rule (<code>{9} < {1,1}</code>)
sequential	element-wise, terminated so a prefix sorts before its extensions
map / set	keys sorted before encoding, so two equal collections built in different insertion orders encode identically

number	one tag for int and float, because <code>(= 0 0.0)</code> is true and <code>(compare 1 3.14)</code> is <code>-1</code>
bignum	length-prefixed, with complemented length and inverted magnitude for negatives
pid / port / ref / fun	faithful round trip, stable arbitrary order — the BEAM promises no more than that either

Verified as three exhaustive properties over 1,225 pairs (`test/bl/datom/terms_test.bl`):

1. the codec's byte order equals the language's `compare`, for every pair
2. equal values encode identically — or an index holds two keys for one value
3. different values never collide — or one silently overwrites the other

3.2 Which found three defects in the language itself

Writing an encoder needs an oracle, and the oracle was broken:

- `compare` reported **distinct** tuples as `EQUAL`. Anything past its known ranks fell to a clause returning 0, so `sorted-set` could silently hold one member where two were inserted (BUG-018).
- `(compare [] {})` was 0 while `(= [] {})` was false — the same defect one type family up.
- `=` and `compare` disagreed about vector-vs-list, and a lazy seq equalled a vector while a list did not (BUG-019).

All three are fixed. The codec is faithful to `=` and `compare`, and those two now agree with each other.

4 The four indexes

The same datoms, sorted four ways. Which one serves a query is decided by which positions are already bound, and that decision is the planner's whole job.

index	sorted by	answers
EAVT	e a v tx op	everything about entity 42 → <code>entity</code> , <code>pull</code>
AEVT	a e v tx op	every <code>:person/name</code> in the database → column scans
AVET	a v e tx op	who is named Ada? → unique lookups, ranges
VAET	v a e tx op	what points at entity 42? → reverse references

AVET and VAET cost an extra write per datom, so they are maintained only for attributes that ask `(:db/index, :db/unique, :db.type/ref)`.

4.1 Keys are binaries, not tuples

Erlang orders tuples by **arity first**, so `{1, 2}` sorts after `{1, 2, 3}` is false — but more importantly, a tuple key cannot support prefix range scans at all: the prefix has a different arity than the keys it should bound, so the scan returns nothing rather than a subset. That is not a performance problem, it is a correctness one, and it was found the hard way.

Binaries compare byte-wise, so a proper prefix sorts immediately before every key that extends it. `key-prefix` plus `prefix-end` (which appends `0xFF`) is therefore a valid scan window. This is the same technique FoundationDB's tuple layer uses.

4.2 The codec must project the language's order, not invent one

An order-preserving encoding is easy to get subtly wrong, and every mistake is silent. Four found by review, each of which corrupted an index without erroring:

integer overflow	encoding shifted by 2^{63} without a range check, so 2^{63} and -2^{63} collided at the same key
-------------------------	--

embedded NUL	strings terminate with 0x00, so a value containing 0x00 could forge a component boundary: "a\\0Pfoo" encoded identically to two components. Escaped as 0x00 0xFF, FoundationDB's solution, which preserves order because 0xFF sorts above every byte a real continuation can begin with
float ordering	IEEE-754 negative floats sort backwards as raw bytes; needs a sign transform
unknown types	fall back to <code>term_to_binary</code> , which does not preserve logical order — so an attribute with no declared <code>:db/ValueType</code> must be refused an AVET index rather than silently misordering it

5 The database as a value

(`db conn`) is $O(1)$. It captures a **basis** — a transaction number — not data.

```
(def snapshot (db conn))
(transact! conn [...])      ; the world moves on
(q '[...] snapshot)         ; snapshot answers exactly as before
```

A held database value is immune to later transactions, forever. This is not snapshot isolation implemented at some cost; it is the trivial consequence of never mutating anything. The value is a small record holding the store, the schema, a basis, and a filter mode.

5.1 Reads are a filter over an append-only log

Because every historical datum is still in the indexes, a read must **decide what is currently true**:

1. scan the index for candidate datoms
2. discard anything after the basis
3. discard anything retracted at or before the basis

Step 3 is where a defect lived that no test caught for two waves. The original implementation asked the store “is this fact retracted?” for **every datum it filtered** — so reading N datoms cost N scans. Quadratic, and measurably: 300 entities took 57ms to filter against 5ms to scan raw, and a reviewer measured 45 **seconds** at 1000 values. Since the join calls this in a loop, every join was quadratic too.

It is now one pass: build a map of fact $\rightarrow$ latest retraction transaction, then filter. Linear.

5.2 A same-transaction retraction wins

Superseding a cardinality-one value emits the retraction of the old value at the same transaction that asserts the new one. A reader pairing them with $\leq$ treats the assertion as surviving its own retraction, and the attribute holds two values.

But $<$ alone is not right either, because a transaction can retract **and then re-assert** the same value, and `tx` cannot order two operations inside one transaction.

The answer is not a better tiebreak. It is to **never write the contradiction**: a transaction records only its NET effect.

```
[[:db/add e :age 37] [:db/add e :age 38]]
naively: assert 37, retract 36, retract 37, assert 38
net:      retract 36, assert 38          (37 never existed)

[:db/retract e :n 7] [:db/add e :n 7]]
naively: retract 7, assert 7
net:      nothing                      (nothing changed)
```

A transaction is atomic, so its intermediate states never existed to any observer. Recording them is not merely wasteful — it is **false**, history describing moments that did not happen.

6 Transactions

Transaction data is data. Maps and operation vectors mix freely:

```
(transact! conn
 [{:db/id -1 :person/name "Ada" :person/email "ada@example.com"}
  [:db/add ada :person/age 36]
  [:db/retract ada :person/nickname "A"]
  [:db/retractEntity old-id]
  [:db/cas ada :person/age 36 37]
  {:db/id :db/current-tx :audit/by "alice"}])
```

6.1 Tempids and upsert

A negative `:db/id` is a placeholder scoped to the transaction; the report says which real id it became. If a tempid asserts a `:db.unique/identity` value that some entity already holds, it **resolves to that entity** rather than creating a new one — which is what makes a transaction idempotent, and what makes “insert or update” a single operation with no race.

Two tempids claiming the same identity resolve to the **same** id, even when neither exists yet. Without that, transacting two maps with one email created two entities sharing a value declared unique: a database violating its own schema, silently.

6.2 The two forms that read

`:db/add` and `:db/retract` say exactly what they do. Two forms do not, and must consult the database to know what they mean:

`[:db/retractEntity e]` retracts every datum the entity holds **and every reference to it**. The incoming half is easy to forget and impossible to repair later — a retracted person whose `:person/friend` references survive leaves dangling pointers that every reader must defend against.

`[:db/cas e a old new]` asserts only if the current value is `old`. Without it, a read-modify-write silently loses a concurrent writer: both read 41, both write 42, one increment vanishes with no error anywhere. `old` of `nil` asserts the attribute is empty, which is “create exactly once”.

6.3 The transaction is an entity

A transaction id is an opaque increasing integer. (`as-of db 1000042`) is answerable but not **askable**: nobody chose 1000042 and nobody can say what it means.

So every transaction records a fact about itself — `:db/txInstant` — as an ordinary datum. And because it is an entity, a caller can annotate it:

```
[:find ?who ?why
 :where [?e :account/balance 175 ?tx]
        [?tx :audit/by ?who]
        [?tx :audit/reason ?why]]
```

Fact → change → author, in one query. There is no audit table and no trigger: the audit trail is what the data model already was, once the transaction became addressable.

7 Datalog

A query is a vector.

```
[:find ?name ?age
 :in $ [?city ...]
 :where [?e :person/city ?city]
        [?e :person/name ?name]
        [?e :person/age ?age]
        [(> ?age 21)]]
```

7.1 A join is a shared variable

There is no JOIN keyword because there is no join **construction**. Two clauses that mention the same variable must agree on its value; that is the entire mechanism. Following a reference is the same thing:

```
[?e :person/name "Alan"]
[?e :person/friend ?f]      ; ?f bound as a VALUE here
[?f :person/name ?friend]   ; and as an ENTITY here
```

Direction is not a property of the data. Asking “who considers Ada a friend” changes only which position the variable appears in — no denormalization, no second index maintained by the application.

7.2 :in, and why four binding forms

?x	scalar	one value
[?x ...]	collection	one row per value — this is how you write IN
[?x ?y]	tuple	destructure one value
[[?x ?y]]	relation	a whole table passed as an argument

The collection form saves WHERE city IN (...) from being built by string assembly or run N times. The relation form is how a query joins against data the database does not hold: ids from another service, rows from a CSV, the results of a previous query.

7.3 Negation, disjunction, and their scoped forms

:or requires its branches to bind identical variables, because a row present in one branch and absent from another has no meaning. That rules out the common case — two branches reaching the same entity by different routes:

```
[ :or-join [?person]                ; only ?person escapes
  [?person :via/email ?x]           ; ?x and ?y are local
  [?person :via/phone ?y]]
```

not-join similarly scopes negation, so “this customer has no order” is sayable without introducing the order to the outer query.

7.4 The planner

Clauses are reordered so that each runs with as much already bound as possible. The heuristic is greedy and deliberately not a cost model: bind the cheapest clause first, where cheapness is how many positions are already known.

One defect here produced **wrong answers rather than slow ones**. Readiness was judged by whether every variable a clause mentioned was bound — but a function clause [(***** ?a 2) ?d] **binds** ?d, so it was never ready, fell to a source-order fallback, and any clause reading ?d ran first against an unbound variable, discarding every row. The query answered #{} and answered correctly once the clauses were written in a different order, which is precisely the coupling a planner exists to remove.

8 Pull

Datalog returns tuples of scalars. An application usually wants a tree.

```
[ :person/name
  { :person/friend [ :person/name ] } ; nest through a reference
  { :person/manager ... }             ; recurse to any depth
  { :person/report 3 }                 ; or a bounded depth
  :person/_friend                     ; REVERSE: who refers to me
  (default :person/nickname "none")
```

```
(limit :person/friend 10)
(:person/name :as :name)]
```

Because the pattern is data, the shape is a **parameter**. A UI knows which fields it is displaying, and that list is a pull pattern — the over-fetching problem that motivates GraphQL is answered by making the shape an argument.

8.1 Recursion and cycles

... descends until nothing new is reachable; an integer bounds the descent. A hierarchy of unknown depth — an org chart, a comment thread, a category tree — cannot be expressed by a fixed pattern at all.

Cycles are ordinary rather than exotic: mutual friendship is one, and so is any bidirectional edge. A revisited entity comes back as a bare `{:db/id N}`. The visited set tracks the path **from the root**, not a global set, because two siblings may legitimately reference the same entity and each should expand it.

8.2 What pull refuses

entity and pull both refuse a **history** view. A history database holds every value an attribute ever had, so folding it into a map would report an entity with three simultaneous ages — a value that existed at no point in time.

That is a category error, not a rare edge, so it fails loudly rather than returning a plausible-looking map describing a state that never was. A wrong answer nothing about which looks wrong is the worst outcome available.

9 Time

```
(as-of db t)      ; the world as it was — inclusive of t
(since db t)      ; only what changed — exclusive of t
(history db)      ; every datum, both kinds
```

as-of is $O(1)$ to construct and copies nothing: the historical value shares the same store and differs only in its basis. Every query and every pull works against it unchanged. There is no `FOR SYSTEM_TIME`, no audit table, no separate API — **the query does not know it is looking at the past**.

as-of being inclusive and since exclusive means the two partition history exactly: every datum falls in one half or the other, never both, never neither. That property is asserted as a test, and it caught a defect where history cleared any existing bound and both views returned everything.

9.1 The actionable forms

```
(changes db e a)    ; every value the field has held, oldest first
(value-at db t e a) ; what it held at t
(tx-of db e a v)    ; when did this become true?
```

changes had an ordering defect worth recording: sorting by tx alone left same-transaction datoms in **index** order — which is by value — so a balance history could read 175 **began**, then 250 **ended**. The change described backwards. Within one transaction a retraction always precedes the assertion that supersedes it.

10 Schema

Schema is data, and it lives in the database. An attribute is an entity whose own attributes describe it:

```
(connect [{:db/ident :person/name
           :db/valueType :db.type/string
           :db/cardinality :db.cardinality/one}])
```

Written as ordinary datoms by an ordinary transaction, so `[?a :db/ident ?name]` answers, `pull` works on an attribute, and a schema change appears in history with a transaction and a timestamp.

10.1 The bootstrap

Schema datoms need a schema to validate against — the circularity every self-describing system meets somewhere. The answer is a fixed set of primordial attributes (`:db/ident`, `:db/valueType`, `:db/cardinality`, `:db/unique`, `:db/index`, `:db/doc`, `:db/txInstant`) installed into every schema. Everything else is ordinary data.

10.2 A known limitation, stated

Reads consult a schema **map** on the database value, not the datoms. The map is touched on every datom filter — cardinality decides whether a value collapses, index selection decides where a datom is written — so deriving it per read would be a serious regression.

The datoms are the record; the map is an index of it. Where they can disagree — a schema change mid-history — the map is the one that is current, so **a historical view reads old data through today's schema**. If an attribute's cardinality changed, a historical read interprets old datoms under the new rule. Filed as FEAT-010.

11 The storage port

Everything above L1 is pure logic over an ordered key/value space. The port is six methods:

```
(-get store k)           ; one key
(-range store lo hi)     ; ordered scan, [lo, hi)
(-put store k v)
(-delete store k)
(-update store k f)       ; atomic read-modify-write
(-commit store ops)       ; apply many operations, in order
```

A substrate must provide: **ordered keys**, **prefix range scans**, **atomic multi-key commit**, and **compare-and-swap**. Anything offering those four can carry this database unchanged.

11.1 The port is proven, not assumed

A port with one implementation is a hypothesis. `priv/datom/store-map.bl` is a second one — a sorted map in an atom, sharing no machinery with ETS: no tables, no process ownership, a different concurrency story. The entire database runs on it unchanged.

`test/bl/datom/conformance_test.bl` runs twenty tests against **both** implementations, and asserts they give identical answers. It earned its keep on its first run: the two stores disagreed about range bounds, because the second implementation had assumed half-open where the port specifies inclusive. That is the single easiest property for a backend author to get wrong, and it now fails loudly rather than silently returning one datom too few from every scan.

This suite is what makes the next backend safe. Run it against a new store and “it works” has a definition rather than being a hope.

11.2 What the ETS implementation does not provide

Stated rather than buried, because these are exactly what a real substrate is for:

- `-commit` is not atomic for a mixed put/delete batch. A process killed mid-batch leaves it half-applied, and a torn index is a corrupt database.
- `-update` is atomic only by single-writer discipline, not by the store.
- There is no durability at all. It is an in-memory development substrate.

The connection is likewise a record doing read-modify-write, so transactions are atomic **only under a single writer**. This is documented at the top of `conn.bl` rather than discovered later.

12 Measurements

12.1 Why counts, not milliseconds

Timings measure the machine as much as the code. While these were taken the host sat at load average 16 with 48 GB of swap in use, and two identical runs of the same benchmark differed by 2.2×.

Store **operations** are exact: they do not move with load, need no warm-up, and — the point — they are what decides what happens when the store stops being local. Every -range is one round trip against a networked backend.

12.2 Reads

Measured by wrapping the L1 port in a counting decorator, which is only possible because L1 is a protocol.

operation	N=10	N=40	shape
entity (point read)	1	1	constant
column scan	1	1	constant
two-clause join	2	2	constant
three-clause join	3	3	constant
pull (2 attributes)	3	3	constant
aggregate (count)	1	1	constant
as-of column scan	1	1	constant

Every read is constant in store operations. Reading the past costs exactly what reading the present costs, which is the db-as-value claim stated as a number.

12.3 The join was not always constant

It evaluated each clause once **per binding row**: 11 ranges at N=10, 41 at N=40. Against ETS a range is a local call and nobody notices. Against a networked store, a join over 1000 entities is **1001 round trips** — at 0.5 ms RTT, half a second of latency for data that fits in one packet.

Each pattern is now scanned once for the whole binding set and unified in memory. This is the single most important number here for choosing a backend, and it is asserted as a test so a regression shows up as a failing count rather than as a slow benchmark somebody has to interpret.

12.4 Range predicates read their window, not the column

A comparison on an AVET-indexed attribute lowers into scan bounds. Measured over 100 entities, counting datoms **returned** rather than ranges issued — a bounded scan and a full scan are both one range, so this is invisible in range counts:

query	answers	datoms read
indexed (> ?a 95)	4	5
indexed, no predicate	100	100
unindexed (> ?a 95)	4	100
two-sided window	4	6

The predicate still runs after the bounded scan. The bound narrows what is read; it does not replace the test — so an off-by-one in the bounds can only make a query slower, never wrong. That is the property that makes the optimisation safe to have, and the acceptance test asserts the indexed and unindexed paths give **identical** answers rather than asserting a speed.

12.5 Writes

entities	ranges	updates	commits
10	62	11	1
20	122	21	1
40	242	41	1

Linear, not constant — a transaction must check what each entity currently holds. Doubling the transaction doubles the work, which is the property that makes bulk loading possible at all. One commit per transaction, whatever its size.

12.6 Language-level costs found along the way

Profiling the database found three defects in beam-lisp itself, each affecting every program in the language:

into was a pessimisation	it took the transient path unconditionally, but transients pay a fixed setup cost plus <code>Process.get/put</code> per element — measured $13\times$ slower at 500 elements, $88\times$ at 2000, and quadratic when called per element in a reduce
vec built element by element	<code>mapv</code> and <code>filterv</code> are <code>(vec (map ...))</code> , so every eager traversal in the language paid a $3\text{--}7\times$ penalty; building in one pass fixed it
the codec escaped NUL per byte	3.1 s to encode 800 datoms against 5 ms for the ETS inserts they became — 99% of write time. Replaced with binary BIFs and a <code>match</code> fast path for the common no-NUL case

13 What a review found that tests did not

Seven review rounds, 51 defects. The distribution is the interesting part: almost none were crashes.

silently wrong	wrong answers with no error: VAET consulted for non-reference attributes, duplicate clauses dropped, unbound predicates passing, a function clause scheduled before its input
silently lost	data that vanished: datum key collisions, an intermediate value written as history, a tempid leaked into durable data as a negative id
silently accepted	malformed input taken as valid: heterogeneous <code>:or</code> branches, empty combinators, non-integer entity ids, <code>:db/index</code> without a value type
stale claims	documentation describing behaviour the code no longer had — the most common category, and arguably the worst, because a header that gets believed is worse than no header

Two defects **cancelled each other into a passing test**: the datum key collision destroyed one of two datoms before the reader's $\leq$ bug could misjudge them. Fixing the writer turned the reader's bug red.

14 The database and the scheduler

Datomic's central claim is that a database is a `VALUE`: immutable, safe to hold, safe to pass around. On the JVM that is mostly about safety — you still have threads, and threads sharing a value is the absence of a problem rather than the presence of a capability.

On the BEAM it becomes something more. A process is cheap enough to make one per request. Isolation is the default. You may share this safely turns into every one of ten thousand processes can hold its own consistent view, and none can interfere with another.

That is not a feature either system lists, because neither can see it alone.

14.1 What the pairing makes possible

- A snapshot per process, at no cost. (`db conn`) captures a basis, not data, so 200 processes taking 200 views copy nothing. There is no connection to pool, because a db value is not a resource.
- A process registry that remembers. A pid is a term, terms are values, and `:db.type/term` makes one an indexed, uniquely-identifying attribute — so `what is this worker doing?` and `what WAS it doing?` are the same query against the same data.
- Supervision that restores rather than repairs. A crashed process held a reference to an immutable value; the restarted one takes another. There is nothing to fix because nothing was mutated.
- A migration with a `before`. `code_change/3` is handed the old state and asked to produce the new one, and the hard part has always been knowing whether it was right. Here the previous version is still addressable.

Let it crash and never destroy a fact turn out to be the same instinct approached from opposite directions: a failure is information, so record it rather than overwrite it.

14.2 The writer

Readers were never the risk. Writers sharing a connection are, and the BEAM makes that case ORDINARY rather than exotic.

Writes are serialized through one process per store. The BEAM answer to a race is not a mutex, it is a process that owns the thing. Reads do not go through it — `db` reads an atom and never blocks — which is the correct asymmetry for a database whose central claim is that a database is a value.

Four details that are not incidental, each of which was a reproduced defect before it was a decision:

it belongs to the store	keyed by handle, two connections over one store were two databases wearing one store: a write through either was invisible to the other
every mutation, not the obvious one	<code>install-schema!</code> is also a read-modify-write. Outside the writer, 80 concurrent installs all returned OK and 74 survived a reopen
re-entrancy	<code>install-schema!</code> ends by transacting, so the writer called itself. The guard is not a workaround: the guarantee is one writer at a time, and this IS that writer
death, and errors	the pid lived in an immutable field, so a replacement was started that nothing called. And an exception raised inside the writer killed it, losing the structured problem list and breaking the NEXT caller

14.3 A note on writing examples

Both concurrency tutorials were written to demonstrate finished work. The first one to spawn a process found a bug in ten seconds: twenty transactions committed, twenty acknowledged, sixteen visible.

Nothing was lost — every datum was on disk. The basis was updated with `reset!`, so a slower transaction holding a LOWER id could land last and rewind the high-water mark. The caller held a success report that was true and a later read that disagreed with it.

An example uses an API the way its NAME invites, which is not how its tests use it. That is the third time in this project that writing a tutorial found defects a passing suite did not.

15 What the last review found

Two reviewers ran against the substrate changes. Both returned “incorrect” with high confidence, and both were right — the 1,225-pair property test used only scalars and simple composites, so it never reached the shapes that broke. Seven defects, and the pattern in them is worth more than the list.

mixed numerics past 2^{53}	splitting large integers into their own tag assumed a bignum exceeds every float. <code>1.0e20</code> is larger than $2^{53}+1$. Tag order is not numeric order
non-uniform encoding shape	-2^{53} took a short encoding and $-2^{53}-1$ a long one, so their comparison ended at the length difference, backwards
sorted collections	a sorted-set is a struct, fell to the map branch, and crashed the encoder outright
lazy seqs	encoded by their unrealized thunk — neither stable nor comparable, so two equal seqs landed at different keys
set vs sequence equality	<code>(= #{1 2} (map identity [1 2]))</code> was TRUE while <code>(= #{1 2} [1 2])</code> was false. Same set, same elements, opposite answers
CAS could not report failure	it returned the value at the key, so a failed swap was indistinguishable from a successful one whenever the key already held <code>new</code> — precisely the case a retry loop exists to detect
the retry loop gave up	a 100-attempt ceiling was reached under ten concurrent writers, and callers got an exception instead of a slower answer

Two more were found by following the review’s leads rather than by the reviewers themselves:

- Deleting the last `use Rustler` removed the only thing that BUILT the crate. The checked-in `.so` kept working until it was deleted, and then the NIF simply stopped existing — with no error except `available?` quietly answering false.
- An AOT build silently lost its native backend entirely (BUG-021). The host module is created at runtime, so AOT never wrote it to disk. An AOT-compiled deployment started without durability, and the only symptom would have been data missing after a restart.

15.1 The shape of these

Almost none were crashes. They were **silent disagreements** — between `=` and `compare`, between an encoding and the order it claims to project, between what a CAS returns and what it means, between what runs in development and what an AOT build contains.

A bounded retry loop deserves particular note, because it looked prudent. A CAS loop that gives up has not made the operation safe, it has made it unreliable — and it cannot spin forever anyway, since every failed iteration means another writer **succeeded**. The system makes progress even when one caller does not.

16 Schema migrations

Growing a schema is safe: a new attribute breaks nothing, because no existing datum mentions it. The dangerous changes are the ones that **tighten** an invariant — “email is now unique”, “role is now

single-valued”. Whether such a change is safe is not a fact about the schema. It is a fact about the **data**. “Email is unique” is fine on a database where emails happen to be unique and a corruption on one where they are not.

And the database that decides is **production**, whose data shapes a developer’s laptop has never seen. A migration that passes every local test can still break against the one database that matters. This is the problem `datom.migrate` exists to solve, and it can only be solved because two properties of the system below it were already true.

16.1 A migration is one value

```
{:migration/id      :2026-08-26-unique-email    ; ordered, immutable name
 :migration/grows   [{:db/ident :person/email    ; the schema delta (data)
                     :db/valueType :db.type/string
                     :db/unique   :db.unique/identity}]
 :migration/checks  [{:check :unique :attr :person/email}] ; the invariant
 :migration/fn      (fn [db] (dedup-unique db :person/email))) ; the remediation
```

The schema delta, the invariant it tightens, and the remediation that makes that invariant true travel together, because they are one decision. `:grows` is ordinary transaction data. `:checks` are questions only the data can answer, expressed as queries. `:fn` is a function of a database value returning the retractions that fix the invariant — a **transaction function**, run in the writer against the transaction’s own before-value.

16.2 plan: walk production before you deploy

Because a database is a **value**, previewing a migration against production is a read. `plan` takes a database value — a local one, or one pulled from a production replica — and reports every way the migration would break against the data actually present, naming the offending entities. It commits nothing; it cannot, because a value has nowhere to commit to.

```
(datom.migrate/plan prod-db migration)
;; => {:status :breaking
      :breaking [{:check :unique :attr :person/email
                  :value "dup@x" :entities [7 8]}]
      ...}
```

This is the feature the value model uniquely affords. A place-oriented migration tool cannot preview a data-dependent constraint against production without running it, because its migrations are not values and its database is not one either. Here it is the deploy gate: run `plan` against production in CI, and a breaking migration is a non-zero exit before anything ships — caught by data shapes the local database has never had.

16.3 apply!: remediate, re-check, grow — atomically

Applying a tightening migration to messy data is a sequence that must not tear: first make the invariant true (the remediation), then enforce it (the grow), then record that it happened (the ledger). If a crash could land between the remediation and the record, a resumed migration would not know it had already fixed the data.

So `apply!` folds all three into **one transaction**. The remediation’s retractions, the schema deltas (written as tempid maps that **upsert** on `:db/ident`, so re-running creates no duplicate attribute entity), and the ledger stamp commit as a single unit — atomic on `redb` by its `WriteTransaction`, serialized by the single writer. Either the whole migration committed or none of it did.

Before committing, `apply!` previews the remediation through `with` (the speculative transaction described below) and re-checks the invariants on the result. If the remediation leaves any invariant still broken — the migration author’s assumption about the data was wrong — it **refuses**, returning `:blocked` with the residual violations rather than committing a schema its data violates.

16.4 The ledger is a query

Which migrations have run is answered by `[?m :migration/id ?id]`, not by a `schema_migrations` table beside the data. A database whose premise is one uniform store should not keep this record somewhere else — and a query-derived ledger is automatically consistent with the transaction that wrote it, because it **is** that transaction’s `datom`. Idempotence and crash-resume follow for free: applying an already-recorded migration is a no-op, and a reopened database recovers the ledger from `datoms` like everything else.

16.5 The two primitives underneath

Migrations added almost no new machinery. They rest on two small, general primitives, each of which was already half-built in the layers below.

with — a **speculative transaction**. `(with conn tx-data)` runs the exact pipeline `transact!` runs, but commits into an **overlay** of the store (a read-through view whose writes land in a private buffer the base never sees) and advances no basis. It returns the database value that **would** result. The writer path already built that value; **with** is that path minus the commit. It needed one insight to be cheap: `datom` writes are all `:put` of new keys — a retraction is a new key, never a delete — so the overlay only ever **adds** keys and never has to mask the base. Zero copy, and a dry-run transfers only the columns it reads.

`:db/fn` — a **transaction function**. `[:db/fn f & args]` in transaction data runs `f` against the before-value and splices its output into the same transaction, which then flows through the identical resolve, validate, realize and commit path as hand-written ops. A transaction function is a macro over transaction data, not a second write path; its `datoms` are validated like any others and commit atomically with them. This is what lets a migration’s remediation and the record of it share one commit.

16.6 One bug the atomicity surfaced

Folding a remediation and a schema grow into one transaction exposed a latent defect in index backfill. When an attribute gains an index, its existing `datoms` must be written into the new index. The backfill read them with a **raw store scan** — which returns tombstoned assertions, because a retraction appends rather than deletes. So a migration that retracted a value and indexed the attribute in the same commit had its retraction silently **undone** by the backfill that followed: the raw scan re-indexed the value the transaction had just retracted. The fix is to backfill from a basis- and retraction-filtered database value, not the raw store — the same filter every other read already goes through. The bug predated migrations; only a change that retracts and indexes at once could reach it.

17 What is not implemented

Stated plainly, because a list of what a system does not do is more useful than a list of what it does.

- **Recursive rules** (`%`), Datomic’s `[(ancestor ?a ?b)]` — the query language has no user-defined rules at all.
- **Multiple data sources** — one `$` only, so no cross-database joins.
- **Schema as-of**. A historical view reads through today’s schema (FEAT-010).
- **Durability**. There is no persistent substrate yet — which is the point of the next section.
- **Multi-writer safety**. Single-writer discipline only.

18 Choosing a substrate

What the storage layer must provide is small and precise:

1. **Ordered keys**, compared byte-wise over binaries.
2. **Prefix range scans**, `[lo, hi)`, returning `key/value` pairs in order.

3. **Atomic multi-key commit** — a transaction’s datoms reach every index together or not at all, because a datom present in EAVT but missing from AEVT is a corrupt database rather than a slow one.
4. **Compare-and-swap**, for id allocation and for `:db/cas`.

Everything else — the data model, the query engine, time travel — is already written and does not care.

18.1 What the measurements imply

Reads are $O(1)$ in store operations, so a networked store costs one round trip per read **regardless of database size**. That is the good news, and it is the result of fixing the join.

Writes are $O(n)$ in the size of the **transaction**, not the database. A bulk import of 10,000 entities is roughly 60,000 range reads today. Against a local store that is fine; against a networked one it is the thing to fix first, and the fix is known: a transaction knows all its entities up front, so the per-operation lookups can be hoisted into one range over the affected span.

18.2 The choice, and what was built

redb via Rustler, with RocksDB TransactionDB as the maturity-first alternative. The full comparison across eight candidates is in FEAT-014; the deciding requirement was atomic multi-key commit, and the axis separating the two finalists is production evidence versus integration simplicity.

Rejected with reasons: sled (unmaintained since 2021, self-described beta), `:dets / :disk_log` (no ordered disk storage in OTP at all), `:mnesia` (wrong abstraction; `ordered_set` unsupported for `disc_only_copies`), and Hobbes — which is pre-alpha and says so. Read at source, Hobbes scores 2 of 4: its `read_range` uses an **exclusive** upper bound where this port specifies inclusive, it has no CAS, and its BTree path writes bytes without `fsync`. Its `Enc.Tuple` is a genuinely order-preserving binary encoding that converged independently on nearly the same design as this codec, which is a good sign for both.

`priv/datom/store-redb.bl` implements the port over `native/datom_redb` — one `WriteTransaction` per commit, `Durability::Immediate` with no way to weaken it, CAS inside a single transaction, and dirty-IO scheduling so a filesystem call cannot stall a BEAM scheduler.

18.3 No Elixir in the datom layer

Reaching Rust used to mean hand-writing an Elixir module whose function bodies all read `:erlang.nif_error(:nif_not_loaded)` — a file containing no logic, existing only to declare that certain names are native, and forcing every native capability to enter through Elixir.

`defnative` moves that declaration into the namespace that uses it:

```
(ns datom.store-redb)

(defnative "datom_redb"
  (redb-open 1)
  (redb-get 2)
  (redb-range 3)
  (redb-commit 2))
```

A NIF is substrate. The declaration that one exists is not. The datom layer is now 17 beam-lisp files and one Rust crate, with **zero** Elixir modules of its own.

18.4 Measured, on an actual disk

500 entities, redb, after the optimisation pass:

operation	before	after	factor
bulk write	3486 ms	551 ms	6.3×

two-clause join	5117 ms	155 ms	33×
column scan	67 ms	41 ms	
range predicate	16 ms	14 ms	

And the write path is finally linear — 2.06× on ETS and 2.43× on redb for 2× the data, where it had been 3.3×.

Three fixes, each found only by measuring: the join’s **match** was quadratic even after its scans became constant; **net-datoms** walked the datoms three times over lazy seqs; and **write-datoms** spent more time assembling its batch than encoding its contents.

Three **other** hypotheses were tried and rejected by measurement — a scan memo that made things slower, an **into→reduce** change that did nothing, and an obvious-looking lazy **cons** that was never the cost. “Which function is slow” and “which function looks slow” are different questions.

18.5 The lesson about cost tests

`cost_test.bl` asserts that a two-clause join costs a constant number of store ranges. **That test passed the entire time the join took five seconds.** The scans were constant and correct; the in-memory match after them was quadratic.

Store operation counts are the right measure for a networked backend and say nothing about the work done between the operations. Both measures are needed, and neither file is sufficient alone.

18.6 The honest summary

The port is six methods wide, one of which (`-commit`) carries the only hard requirement: **atomicity across many keys**. redb provides it directly, so `-commit` delegates rather than implements — which is the whole argument for using a storage engine instead of a file.

Three implementations now satisfy the same 24-test conformance suite: ETS, a sorted map in an atom, and redb on disk. A store that provides ordered byte-wise keys, inclusive-bound prefix scans, atomic multi-key commit and compare-and-swap can carry this database unchanged. That claim is no longer an argument; it is a suite that runs.